

Architettura dei Calcolatori

Terzo Parziale – 9 giugno 2021 – 1h45

PARTE 1 – RISPOSTA SINGOLA - Ogni domanda ha una sola risposta VERA.

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata fa perdere il punteggio negativo riportato a fianco della domanda
- Una risposta lasciata in bianco viene valutata 0

1. [2, -0.5] Si consideri di eseguire il seguente stralcio di codice su una pipeline RISC-V con logica di forwarding e con dynamic branch predictor a 1 bit. Inizialmente il predittore è settato a BRANCH NOT TAKEN. Qual è la misprediction rate per questo codice?

```
li    x5, 1
li    x6, 10
loop:
    addi x6, x6, -1
    slli x5, x5, 1
    bgt  x6, x0, loop
```

- a) 10%
- b) 20%
- c) 30%
- d) Nessuna delle precedenti
2. [1, -0.5] Un processore Out-of-Order
- a) Riduce gli stalli della pipeline
- b) Non trae beneficio dal riordino delle istruzioni effettuato dal compilatore
- c) Se non fosse per il suo costo garantirebbe ad ogni nuova generazione di processori un incremento continuo di efficienza
- d) Nessuna delle precedenti
3. [1, -0.5] Rispetto ad un design monolitico una CPU pipelined
- a) Riduce la latenza di esecuzione delle istruzioni
- b) Riduce il throughput del sistema
- c) Consente critical paths più corti, e quindi frequenza più alta
- d) Nessuna delle precedenti
4. [1, -0.5] In un design pipelined
- a) I segnali di controllo sono diversi da quelli del design monolitico
- b) I segnali di controllo richiedono logica di generazione diversa da quella del design monolitico
- c) I segnali di controllo devono essere propagati come i dati attraverso gli stadi
- d) Nessuna delle precedenti

PARTE 2 – (POSSIBILI) RISPOSTE MULTIPLE -
Ogni domanda può avere da una a quattro risposte CORRETTE.

- Ogni risposta esatta viene calcolata: +1
 - Ogni risposta errata viene calcolata: -0.5
 - Una risposta lasciata in bianco viene calcolata: 0
5. Una CPU RISC-V 32bit usa una L1 cache direct mapped grande 256B e con linee da 8 word. Quali affermazioni sono vere?
- a) Ogni linea può ospitare 2^{19} diversi indirizzi
 - b) A cache fredda due accessi successivi agli indirizzi 0x0020 e 0x0033 generano rispettivamente una compulsory miss e una hit
 - c) A seguito degli accessi al punto b) due nuovi accessi agli indirizzi 0x0000 e 0x2021 generano rispettivamente una compulsory miss e una conflict miss
 - d) A seguito degli accessi al punto b) e c) un nuovo accesso all'indirizzo 0x14 genera una compulsory miss
6. Una cache 4-way set associative con campo index e campo offset da 5 bit
- a) È quattro volte più grande di una cache direct mapped con analoghi campi index e offset
 - b) Richiede meno logica di comparazione rispetto ad una cache direct mapped analoga
 - c) A parità di dimensioni ha miss rate medio più basso rispetto ad una cache direct mapped
 - d) A parità di dimensioni ha miss rate medio più basso rispetto ad una cache fully associative
7. La Memory Management Unit
- a) Accelera in hardware la fase di page table walk
 - b) Sfrutta una piccola cache locale per ricordare le traduzioni più recenti
 - c) Contiene tutte le traduzioni per ogni pagina del virtual address space
 - d) Ospita la page table
8. L'architettura di Harvard
- a) Evita per costruzioni alcuni tipi di structural hazards
 - b) È tipicamente implementata al primo livello della gerarchia di cache
 - c) È tipicamente implementata all'ultimo livello della gerarchia di cache
 - d) È meno costosa da implementare rispetto all'architettura di Von Neumann

PARTE 3 – DOMANDE APERTE

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata può causare una penalità che dipende dalla gravità dell'errore
- Una risposta lasciata in bianco viene calcolata: 0

9. [12 pt] Si consideri il seguente stralcio di codice C:

```
int A[100] = {0,1,2,3,...}; // contiene i numeri da 0 a 99
int *p;
p = &A[i] + ((a+b)*(c-d));
*p = A[i];
```

Tradurre le espressioni C in assembly RISC-V, considerando la seguente allocazione delle variabili sui registri:

$A \rightarrow x10$; $i \rightarrow t0$; $a \rightarrow x11$; $b \rightarrow x12$; $c \rightarrow x13$; $d \rightarrow x14$

Oltre ai registri già menzionati il programma assembly risultante deve usare solo t1 e t2, che sono da considerare inizializzati ai valori 0 e 1, rispettivamente.

Assumendo che:

$\&A[0] = 0x00402400$;

$i = 5$;

$a = 3$;

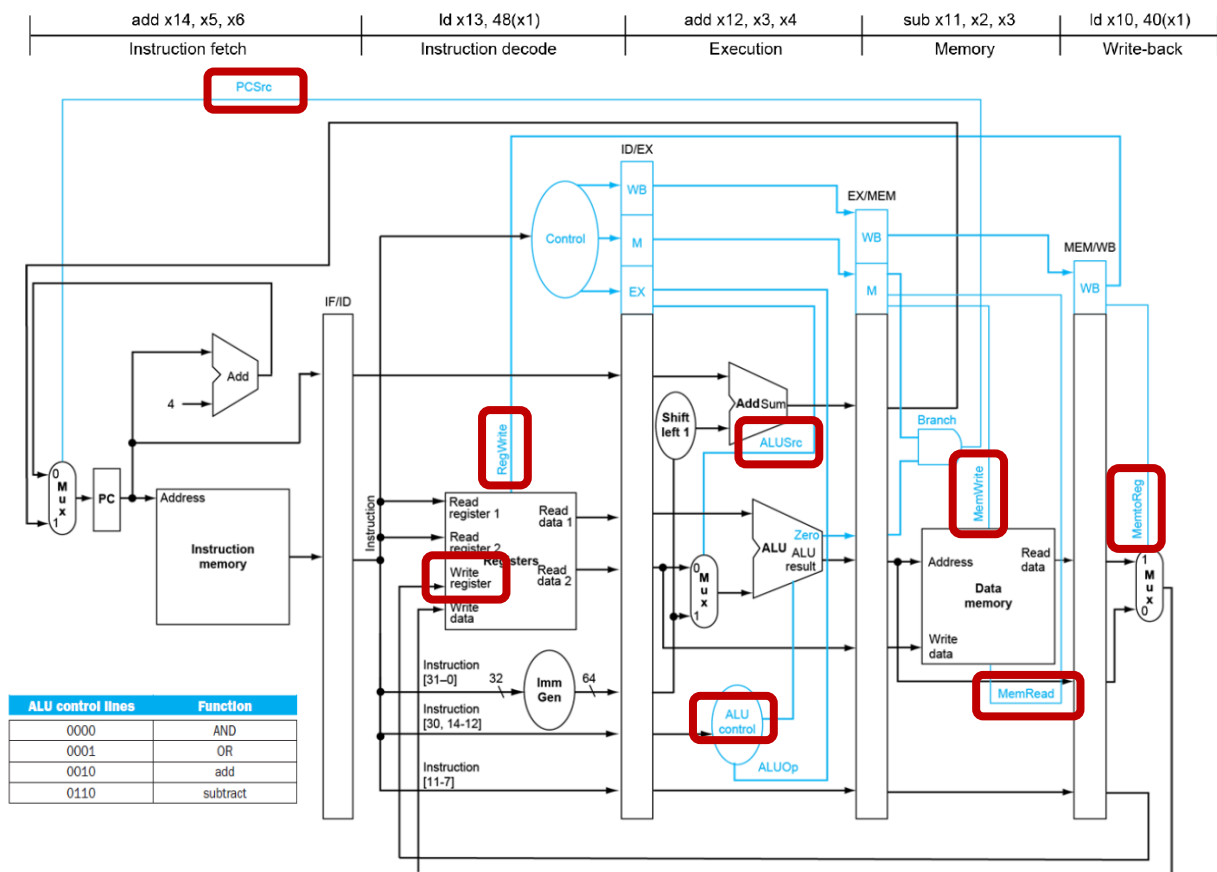
$b = 22$;

$c = 10$;

$d = 9$;

- a) [3 pt] Dire in quanti cicli esegue il programma su una pipeline senza logica di detection/forwarding (si consideri la pipeline vuota). Dire che indirizzo viene inserito nel puntatore p e, se possibile, che valore viene scritto nella locazione di memoria puntata.
- b) [3 pt] Riscrivere il programma per gestire la problematica dei data hazards tramite inserimento di NOP. Quanti cicli impiega il programma così modificato a eseguire sulla stessa pipeline senza logica di detection/forwarding (si consideri sempre la pipeline vuota). Dire che indirizzo viene inserito nel puntatore p e che valore viene scritto nella locazione di memoria puntata.
- c) [2 pt] Se possibile, riordinare le istruzioni del programma al punto b) per ridurre il numero di NOP. Quanto impiega il programma così modificato a eseguire?
- d) [2 pt] Come occorre modificare il programma al punto a) perché esegua senza errori su una pipeline dotata di logica di detection dei data hazard e di forwarding? Quanti cicli impiega il programma così modificato a eseguire su quella pipeline?
- e) [2 pt] Come si possono raggruppare le istruzioni del programma al punto d) perché eseguano correttamente su una pipeline dual-issue? Quanti cicli impiega il programma a eseguire su quella pipeline?

10. [6 pt] Si consideri il seguente diagramma single-cycle



relativo all'esecuzione del seguente stralcio di codice assembly:

```
ld    x10, 40(x1)
sub   x11, x2, x3
add   x12, x3, x4
ld    x13, 48(x1)
add   x14, x5, x6
```

Dire quanto valgono i segnali cerchiati in figura, motivando chiaramente le risposte.